SoilFlow 1D — Beta 1

Develped by Mostafa Ghasemzadeh Ortakand

Email: mortakand@Gmail .com , mghasemzadeh@ut.ac.ir

Software Documentation

What Is SoilFlow 1D?

SoilFlow 1D is a desktop simulation tool for modeling water flow, solute transport, and heat transfer through layered soil profiles. It is built on the physics of the Richards equation — the standard mathematical model for variably saturated (partially wet) soil — and is designed to be a Python-based, open alternative to tools like Hydrus-1D.

It runs entirely on your local machine with a graphical interface. No internet connection, no cloud, no external services.

Core Capabilities

- Simulate 1D vertical water flow through soil layers
- Model solute (chemical) transport through the soil column
- Model heat transport through the soil profile
- Include root water uptake with stress response
- Define time-varying boundary conditions (rainfall, evaporation, etc.)
- Visualize results as static plots or animated time-lapse profiles
- Export results to CSV and figures to image files
- Save and load projects

Architecture Overview

The software is split into two packages:

core/ → numerical solvers (physics engine)

gui/ → PyQt6 graphical interface

main.py → application entry point

The Physics Engine (core/)

richards.py — Water Flow Solver

The heart of the software. Implements the Richards1D class which solves the 1D Richards equation:

$$\frac{\partial \theta}{\partial t} = \frac{\partial}{\partial z}\left[K(h)\left(\frac{\partial h}{\partial z}+1\right)\right] - S(h,z)$$

where θ is volumetric water content, h is pressure head, $K(h)$ is hydraulic conductivity, and S is the root uptake sink term.

Key internals:

- Builds a vertical node grid from the soil layer definitions
- Uses Picard iteration to solve the nonlinear system at each time step
- Assembles a banded tridiagonal matrix for efficient linear solving
- Applies top and bottom boundary conditions per time step
- Computes water balance: $\Delta S = \text{inflow} - \text{outflow} - \text{uptake}$
- Supports adaptive time stepping with dt_min and dt_max
- Stores output profiles of h, θ, K, and flux at requested output times

Supported top boundary conditions:

- Constant pressure head
- Constant flux
- Atmospheric (rainfall + potential evaporation with ponding logic)

Supported bottom boundary conditions:

- Constant head
- Free drainage

- Seepage face
- Constant flux

hydraulic_models.py — Soil Water Retention Models

Provides the mathematical relationships between pressure head $h$and the hydraulic properties $\theta(h)$, $K(h)$, and $C(h) = d\theta/dh$.

Two models are implemented:

Van Genuchten–Mualem:

$$\theta(h) = \theta_r + \frac{\theta_s - \theta_r}{[1+ \mid \alpha h \mid^n]^m}$$

$$K(h) = K_s \cdot S_e^l \cdot \left[1 - (1 - S_e^{1/m})^m\right]^2$$

Brooks–Corey:

$$\theta(h) = \theta_r + (\theta_s - \theta_r)\left(\frac{h_b}{h}\right)^\lambda \text{ for } h < h_b$$

Both models are registered in a MODELS dictionary so the solver can select them by name from the GUI configuration.

Preset parameter values are provided for common soil textures (Sand, Loam, Clay, Silt, etc.).

root_uptake.py — Root Water Uptake

Computes the sink term $S(h, z)$ representing water removed from the soil by plant roots.

Process:

1. Distributes potential transpiration T_p over depth using a root density function (e.g. exponential decay with depth)
2. Applies a water stress reduction factor $\alpha(h)$based on soil water pressure

3. Sets $S = 0$ for nodes below the defined root depth

Two stress models are supported:

- **Feddes model**: piecewise linear stress function with threshold heads h_1–h_4
- **Van Genuchten stress model**: smooth sigmoidal reduction

solute.py — Solute Transport Solver

Solves the 1D advection–dispersion equation (ADE):

$$\frac{\partial(\theta C)}{\partial t} = \frac{\partial}{\partial z}\left[D(\theta, q)\frac{\partial C}{\partial z} - qC\right] - \mu\theta C + \gamma$$

where C is concentration, D is the hydrodynamic dispersion coefficient, q is the Darcy flux from the Richards solver, μ is a decay rate, and γ is a source term.

Uses the flow field (θ, K, q) computed by Richards1D as input. Supports sorption and first-order decay reactions.

heat.py — Heat Transport Solver

Solves 1D soil heat transport:

$$C_v\frac{\partial T}{\partial t} = \frac{\partial}{\partial z}\left[\lambda(z)\frac{\partial T}{\partial z}\right] - C_w q\frac{\partial T}{\partial z}$$

where T is temperature, λ is thermal conductivity, C_v is volumetric heat capacity, and $C_w q$ is the convective heat flux carried by water flow.

Thermal properties are defined per soil layer.

simulation.py (core) — Simulation Orchestrator

The run_simulation(config, progress_cb) function is the bridge between the GUI and the solvers.

It:

1. Parses the full configuration dictionary from the GUI

2. Constructs Richards1D, and optionally Solute1D and Heat1D
3. Runs the coupled simulation, reporting progress via progress_cb(percent, message)
4. Returns a unified results dictionary with keys:
    - "z" — depth array
    - "times" — output time points
    - "theta" — water content profiles over time
    - "h" — pressure head profiles
    - "K" — hydraulic conductivity profiles
    - "conc" — solute concentration profiles (if enabled)
    - "temp" — temperature profiles (if enabled)
    - "water_balance" — cumulative inflow, outflow, storage change

The Graphical Interface (gui/)

main.py — Entry Point

Starts the Qt application, sets the application name to "SoilFlow 1D", applies the Fusion visual style, and launches MainWindow.

main_window.py — Main Window

The top-level window that connects everything together.

Layout:

- Left panel: tabbed input configuration
- Right panel: live plot canvas
- Bottom: status bar with progress indicator

Tabs in the left panel:

1. Soil Profile
2. Boundary Conditions

3. Time Series
4. Solute Transport
5. Heat Transport
6. Root Uptake
7. Simulation Control

The SimulationWorker(QThread) runs the solver in a background thread so the GUI stays responsive. It emits:

- progress(int, str) — updates the progress bar and status message
- finished(dict) — delivers results to the plot and results panels
- error(str) — displays error messages if the solver fails

Menu and toolbar provide: New Project, Open, Save, Export, Run Simulation.

soil_profile.py — Soil Profile Editor

Defines the soil column with up to 5 layers.

Each layer (LayerPanel) has:

- Thickness (cm)
- Texture class (Sand, Loam, Clay, Silt, etc.)
- Hydraulic model selection (Van Genuchten–Mualem or Brooks–Corey)
- All model parameters: θ_r, θ_s, α, n, K_s, l

Preset parameter values auto-fill when a texture is selected.

A ProfileCanvas draws a live color-coded schematic of the soil column on the right side of the tab, updating in real time as layers are added or modified. Each texture has a distinct color. Depth tick marks and layer labels are drawn using Qt's QPainter.

boundary_conditions.py — Boundary Condition Panel

Configures the top and bottom boundaries of the soil column.

Top boundary options:

- Constant pressure head
- Constant flux
- Atmospheric (uses rainfall and potential evaporation from the time series)

Bottom boundary options:

- Constant head
- Free drainage
- Seepage face
- Constant flux

Settings feed directly into the Richards solver's _apply_bc method.

time_series.py — Time Series Editor

A table-based editor for time-varying inputs. Each row is a (time, value) pair.

Supported series:

- Rainfall rate
- Potential evapotranspiration
- Potential transpiration
- Solute concentration at top boundary
- Temperature at top boundary

Rows can be added, deleted, and reordered. Data is interpolated by the solver at each time step.

solute_tab.py — Solute Transport Configuration

Configures the solute transport module:

- Longitudinal dispersivity
- Molecular diffusion coefficient
- Sorption parameters (linear, Freundlich, Langmuir)

- First-order decay rate
- Initial concentration profile
- Top and bottom concentration boundary conditions

heat_tab.py — Heat Transport Configuration

Configures the heat transport module per layer:

- Thermal conductivity λ
- Volumetric heat capacity C_v
- Initial temperature profile
- Top boundary temperature (links to time series)
- Bottom boundary condition

root_uptake_tab.py — Root Uptake Configuration

Configures the root system:

- Maximum root depth (cm)
- Root density distribution shape (uniform, exponential, linear)
- Stress model: Feddes or Van Genuchten
- Feddes parameters: h_1, h_2, h_3, h_4 (anoxia and wilting point thresholds)

simulation_tab.py — Simulation Control Panel

Controls simulation execution:

- Run and Stop buttons
- Real-time progress bar
- Log window showing solver messages
- Simulation time settings: end time, time step, output frequency

The SimulationWorker here is a lightweight GUI-side worker. The actual numerical work is done by the core run_simulation function called from main_window.py.

plot_canvas.py — Plot Canvas

The right-side visualization panel. Built on matplotlib embedded in Qt.

Methods:

- clear() — resets the axes to a blank state
- plot_results(results, plot_type, time_idx) — plots a single profile (e.g. θvs z) at a chosen time step
- show_plot(plot_type) — convenience wrapper to show the last time step
- animate(results) — runs a FuncAnimation loop cycling through all time steps, showing how the profile evolves over time

Supported plot types: theta, h, K, conc, temp

The y-axis is always inverted so depth increases downward, matching physical convention.

results_tab.py / results.py — Results Panel

Presents simulation output in structured form:

- Variable selector (water content, pressure head, conductivity, concentration, temperature)
- Time step selector
- Water balance summary table (cumulative inflow, outflow, storage change)
- Buttons to trigger static plots or animations via PlotCanvas

project_manager.py — Project File Management

Handles saving and loading complete project states as JSON files. A project includes all input configuration: soil layers, boundary conditions, time series, solute/heat/root settings, and simulation parameters. Results can optionally be saved alongside.

exporter.py — Results Exporter

Exports simulation output:

- Profiles to CSV (one file per variable, rows = depth nodes, columns = time steps)
- Figures to PNG or PDF via matplotlib's savefig

Accessible from the main window toolbar or menu.

validators.py — Input Validators

Provides validation logic used across all input tabs:

- Layer thickness must be > 0
- $\theta_r < \theta_s$, both in $[0, 1]$
- $\alpha > 0, n > 1$ for Van Genuchten
- Time series must be monotonically increasing in time
- Simulation end time must be > 0
- Root depth must not exceed total soil column depth

Errors are surfaced in the GUI before the simulation is allowed to run.

Beta 1 — Known Notes and Limitations

- Solute and heat transport are one-way coupled to flow (flow drives transport, not vice versa)
- The simulation panel in simulation_tab.py contains a placeholder worker for GUI testing; the real solver is invoked from main_window.py
- Animation uses blit=False for stability across matplotlib versions; this is slightly slower than blit mode but avoids axes reference errors
- Project files are plain JSON — human-readable and editable by hand
- No 2D or 3D support; strictly 1D vertical column
- No automatic mesh refinement; node count is fixed at initialization

SoilFlow 1D — Beta 1 Known Issues & Limitations

1. Animation Not Working

The animation button does not produce visible output.

Likely causes:

- FuncAnimation object gets garbage collected before it renders because no persistent reference is held at the window level
- blit=False with certain Qt backends causes silent failure on some systems
- The results dict passed to animate() may have mismatched array shapes between z and data[frame]

Status: under investigation — the animation infrastructure exists but output is not displayed correctly.

2. CSV Export Not Saving Results

Exporting results to CSV produces no file or an empty file.

Likely causes:

- The exporter.py export function may be called before results are fully populated in the results dict
- File path resolution may silently fail without surfacing an error to the user
- Column/row orientation of the output arrays may be transposed, causing a write with zero rows

Status: export function exists but is non-functional in Beta 1.

3. Water Table Calculation Incomplete

The water table depth is not computed or displayed.

What is missing:

- No post-processing step that scans the pressure head profile $h(z)$for the node where $h = 0$(the phreatic surface)
- No water table depth output in the results dict
- No display of water table position in the results tab or plot

The water table depth z_{wt}should be found by interpolating:

$$z_{wt}: h(z_{wt}) = 0$$

Status: not implemented.

4. Temperature Calculation Incomplete

The heat transport module (heat.py) is not fully coupled to the simulation pipeline.

What is missing:

- run_simulation in core/simulation.py may not be constructing or calling Heat1D even when heat transport is enabled in the GUI
- Temperature initial conditions from heat_tab.py may not be passed correctly into the solver
- Temperature boundary condition from the time series is not confirmed to be wired through to Heat1D
- Temperature profiles may be absent from the results dict ("temp" key missing or empty)

Status: heat transport module exists but end-to-end coupling is incomplete.

5. Water Balance Calculation Incomplete

The water balance summary shown in the results tab is incomplete or incorrect.

What is missing:

- Cumulative bottom outflow may not be tracked correctly across all time steps
- Root water uptake is not being subtracted from the balance in all code paths

- The balance error term $\Delta S - (\text{inflow} - \text{outflow} - \text{uptake})$is not computed or reported
- No warning is raised when the balance error exceeds an acceptable threshold (e.g. 1%)

The correct water balance check is:

$$\varepsilon = \Delta S - \int_0^T (q_{top} - q_{bot} - S_{root})\, dt$$

Status: partially implemented — storage change is tracked but flux accounting is incomplete.

6. Pressure Head Validation Missing

Pressure head inputs and outputs are not validated.

What is missing:

- No check that the initial pressure head profile $h_0(z)$is physically reasonable (e.g. not strongly positive in an unsaturated setup)
- No check that the constant head boundary values are within a plausible range
- No runtime warning when $h$diverges or exceeds typical bounds during the Picard iteration
- The GUI does not flag when a user-entered head value is inconsistent with the selected hydraulic model range

Status: validators.py does not currently cover pressure head fields.

7. Concentration Validation Missing

Solute concentration inputs are not validated.

What is missing:

- No check that initial concentration $C_0 \geq 0$everywhere
- No check that the top boundary concentration is non-negative

- No check that dispersion coefficient $D > 0$
- No check that decay rate $\mu \geq 0$
- No warning when the Peclet number $Pe = qL/D$is large (advection-dominated), which can cause numerical oscillation without upwinding

The grid Peclet condition for numerical stability is:

$$Pe = \frac{|\, q \,|\, \Delta z}{D} \leq 2$$

Status: validators.py does not currently cover solute transport fields.

Summary Table

Issue	**Severity**	**Status**
Animation produces no output	High	Under investigation
CSV export not saving	High	Not functional
Water table not calculated	Medium	Not implemented
Temperature coupling incomplete	Medium	Partially implemented
Water balance accounting incomplete	Medium	Partially implemented
Pressure head validation missing	Medium	Not implemented
Concentration validation missing	Medium	Not implemented

These issues are tracked for resolution in the next development cycle toward Beta 2.